

Universidad Nacional de San Antonio Abad del Cusco

Departamento Académico de Informática

COMPUTACIÓN GRÁFICA I

Práctica N° 07

TRANSFORMACIÓN 2D — ESCALA & ROTACIÓN

Pipeline Moderno: Vertex Shader y Fragment Shader

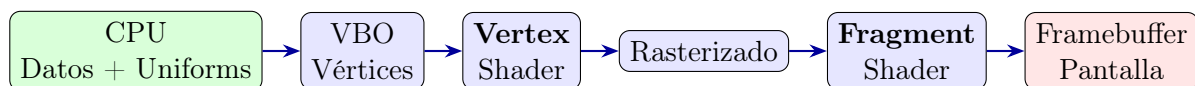
1. OBJETIVO

- Comprender los fundamentos matemáticos de la transformación de rotación 2D.
- Expresar la rotación mediante coordenadas homogéneas y matrices 3×3 .
- Implementar la rotación en el **pipeline programable** de OpenGL moderno usando *Vertex Shader* y *Fragment Shader* escritos en GLSL.
- Comparar el enfoque clásico (OpenGL fijo) con el enfoque moderno (shaders).
- Entender los conceptos geométricos de rotación.
- Implementar y aplicar transformaciones de rotación.

2. BASE TEÓRICA

2.1. Pipeline Moderno de OpenGL

A diferencia del pipeline clásico (OpenGL ≤ 2.1 con `glBegin/glEnd`), el pipeline moderno (OpenGL ≥ 3.3 core profile) **requiere** al menos dos shaders escritos en GLSL:



- **Vertex Shader:** se ejecuta una vez por vértice; aplica la traslación multiplicando la posición por la matriz de transformación enviada como **uniform**.
- **Fragment Shader:** se ejecuta por cada fragmento (píxel) generado; determina el color final.

2.2. Rotación en 2D

Un punto puede ser rotado en el espacio 2D si se especifica el centro de rotación (pivote) como también el ángulo de rotación.

Un punto $p_1(x_1, y_1)$ gira sobre el origen a través de un ángulo theta (θ) para alcanzar otro punto $p_2(x_2, y_2)$. Si p_1 está a una distancia r desde el origen, sus coordenadas se pueden representar como:

$$[r \cos(\alpha), r \sin(\alpha)]$$

Donde alfa α es el ángulo de inclinación de la línea p_0, p_1 con respecto al eje x (Fig. 1.).

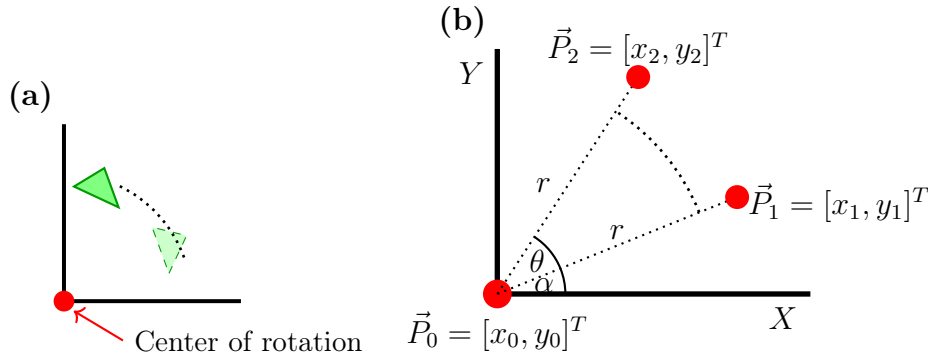


Figura 1: Rotación en 2D.

Después de la rotación, el punto p_2 se puede expresar como:

$$p_2 = [r \cos(\alpha + \theta), r \sin(\alpha + \theta)].$$

Por trigonometría tenemos:

$$\begin{aligned} \sin(\alpha + \theta) &= \sin(\alpha) \cos(\theta) + \cos(\alpha) \sin(\theta), \\ \cos(\alpha + \theta) &= \cos(\alpha) \cos(\theta) - \sin(\alpha) \sin(\theta). \end{aligned}$$

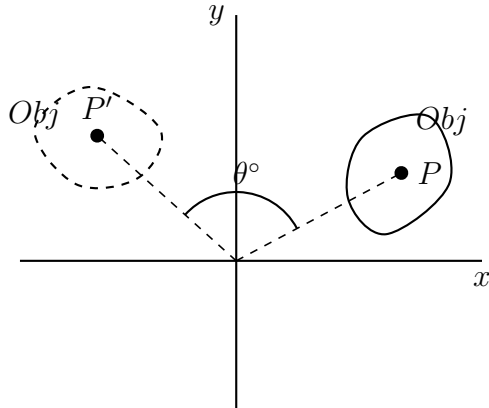
Aplicando esta ecuación a las coordenadas del punto después de la rotación, tenemos:

$$\begin{aligned} \underbrace{\begin{bmatrix} x_2 \\ y_2 \end{bmatrix}}_{\vec{p}_2} &= \begin{bmatrix} r \cos(\alpha + \theta) \\ r \sin(\alpha + \theta) \end{bmatrix} \\ &= \begin{bmatrix} r(\cos(\alpha) \cos(\theta) - \sin(\alpha) \sin(\theta)) \\ r(\sin(\alpha) \cos(\theta) + \cos(\alpha) \sin(\theta)) \end{bmatrix} \\ &= \begin{bmatrix} \underbrace{r \cos(\alpha)}_{x_1} \cos(\theta) - \underbrace{r \sin(\alpha)}_{y_1} \sin(\theta) \\ \underbrace{r \sin(\alpha)}_{y_1} \cos(\theta) + \underbrace{r \cos(\alpha)}_{x_1} \sin(\theta) \end{bmatrix} \\ &= \underbrace{\begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix}}_{R(\theta)} \underbrace{\begin{bmatrix} x_1 \\ y_1 \end{bmatrix}}_{\vec{p}_1}, \end{aligned}$$

R es una matriz 2×2 que representa la operación de rotación para puntos no homogéneos en el espacio 2D.

θ es el ángulo de rotación.

Tener en cuenta que el pivote en este caso es el origen.



Si trabajamos con coordenadas homogéneas, para la rotación de un punto $p_1 = (x_1, y_1, 1)$ a otro punto $p_2 = (x_2, y_2, 1)$ tenemos:

$$\underbrace{\begin{bmatrix} x_2 \\ y_2 \\ 1 \end{bmatrix}}_{\mathbf{p}_2} = \underbrace{\begin{bmatrix} \cos(\theta) & -\sin(\theta) & 0 \\ \sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 1 \end{bmatrix}}_{R(\theta)} \underbrace{\begin{bmatrix} x_1 \\ y_1 \\ 1 \end{bmatrix}}_{\mathbf{p}_1}$$

Después de la rotación, el punto p_2 se puede expresar como:

Donde R es una matriz de 3×3 que representa la operación de rotación para puntos homogéneos en el espacio 2D; y θ es el ángulo de rotación.

Ejemplo 1: Consideremos un segmento de línea que se extiende desde $p_1 = (3, 4)$ hasta $p_2 = (10, 7)$. La línea gira a través de un ángulo de 45 grados sobre el origen.

Calcule las nuevas posiciones de sus puntos finales:

Solución:

Supongamos que los puntos homogéneos p_1 y p_2 se rotan hasta p_3 y p_4 respectivamente. Usando la correspondiente ecuación, las nuevas posiciones se obtienen:

$$\begin{aligned} \underbrace{\begin{bmatrix} x_3 \\ y_3 \\ 1 \end{bmatrix}}_{p_3} &= \begin{bmatrix} \cos(\theta) & -\sin(\theta) & 0 \\ \sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ y_1 \\ 1 \end{bmatrix} \\ &= \underbrace{\begin{bmatrix} \cos(45) & -\sin(45) & 0 \\ \sin(45) & \cos(45) & 0 \\ 0 & 0 & 1 \end{bmatrix}}_{R(45)} \underbrace{\begin{bmatrix} 3 \\ 4 \\ 1 \end{bmatrix}}_{p_1} = \begin{bmatrix} -\frac{1}{\sqrt{2}} \\ \frac{7}{\sqrt{2}} \\ 1 \end{bmatrix} = \begin{bmatrix} -0,7071 \\ 4,9497 \\ 1 \end{bmatrix} \end{aligned}$$

$$\begin{aligned}
\underbrace{\begin{bmatrix} x_4 \\ y_4 \\ 1 \end{bmatrix}}_{p_4} &= \begin{bmatrix} \cos(\theta) & -\sin(\theta) & 0 \\ \sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_2 \\ y_2 \\ 1 \end{bmatrix} \\
&= \underbrace{\begin{bmatrix} \cos(45) & -\sin(45) & 0 \\ \sin(45) & \cos(45) & 0 \\ 0 & 0 & 1 \end{bmatrix}}_{R(45)} \underbrace{\begin{bmatrix} 10 \\ 7 \\ 1 \end{bmatrix}}_{p_2} = \begin{bmatrix} \frac{3}{\sqrt{2}} \\ \frac{17}{\sqrt{2}} \\ 1 \end{bmatrix} = \begin{bmatrix} 2,1213 \\ 12,0208 \\ 1 \end{bmatrix}.
\end{aligned}$$

2.3. Escalamiento General en 2D.

Cualquier operación de escalado debe mantener un punto fijo en su posición original mientras se modifica el resto de los puntos bajo consideración. En el caso general, el punto fijo puede ser cualquier punto arbitrario en el espacio 2D. En este caso, debemos trasladar el objeto bajo escala para que el punto fijo coincida con el origen antes de aplicar las ecuaciones de escalamiento. Luego, la traslación se realiza nuevamente para revertir el efecto de la primera traslación. Esta secuencia de operaciones es similar a la que se implementó en Rotación General.

Por lo tanto, dado un vértice $p_1 = [x_1, y_1]$ y un punto fijo $p_0 = [x_0, y_0]$ podemos expresar la secuencia de operaciones como:

$$\begin{bmatrix} x_2 \\ y_2 \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 - x_0 \\ y_1 - y_0 \\ 1 \end{bmatrix} + \begin{bmatrix} x_0 \\ y_0 \\ 0 \end{bmatrix}$$

donde S es la matriz de escala; s_x y s_y son los factores de escala; $[-x_0, -y_0]$ y $[x_0, y_0]$ son los vectores de traslación; p_0 es el punto fijo; p_1 es el punto anterior al escalamiento; y p_2 es el punto después de escalar.

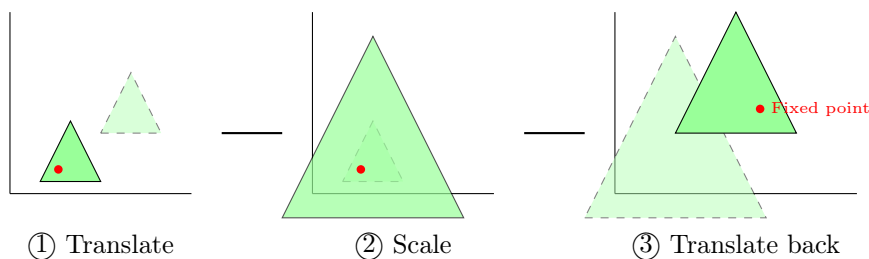


Figura 2: Escalamiento general en 2D.

En coordenadas homogéneas, la operación de escala se aplica como:

$$\begin{bmatrix} x_2 \\ y_2 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & x_0 \\ 0 & 1 & y_0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & -x_0 \\ 0 & 1 & -y_0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ y_1 \\ 1 \end{bmatrix}$$

donde S es la matriz de escala; S_x y S_y son los factores de escala; T son las matrices de traslación, $[-x_0, -y_0]$ y $[x_0, y_0]$ son los vectores de traslación; p_0 es el punto fijo; p_1 es el punto antes de escalar; y p_2 es el punto después de escalar.

3. DESARROLLO DE LA PRÁCTICA

3.1. Código C++ completo — Escalamiento

```

1 #include <GL/glew.h>
2 #include <GLFW/glfw3.h>
3 #include <glm/glm.hpp>
4 #include <glm/gtc/type_ptr.hpp>
5 #include <iostream>
6 #include <string>
7 #include <cmath>
8
9 // -----
10 // Fuentes de los shaders (en crudo como cadenas)
11 // -----
12 const char* vertSrc = R"(
13 #version 330 core
14 layout(location = 0) in vec2 aPos;
15 uniform mat3 uTransform;
16 void main() {
17     vec3 p = uTransform * vec3(aPos, 1.0);
18     gl_Position = vec4(p.xy, 0.0, 1.0);
19 }
20 )";
21
22 const char* fragSrc = R"(
23 #version 330 core
24 uniform vec4 uColor;
25 out vec4 fragColor;
26 void main() {
27     fragColor = uColor;
28 }
29 )";
30
31 // -----
32 // Variables globales
33 // -----
34 GLuint shaderProg, vao, vbo;
35 float factorEscalaX = 1.0f;
36 float factorEscalaY = 1.0f;
37
38 // Funci n auxiliar para revisar errores de compilaci n
39 void chequearErroresShader(GLuint shader, std::string tipo) {
40     GLint success;
41     GLchar infoLog[1024];
42     if (tipo != "PROGRAMA") {
43         glGetShaderiv(shader, GL_COMPILE_STATUS, &success);
44         if (!success) {
45             glGetShaderInfoLog(shader, 1024, NULL, infoLog);
46             std::cerr << "ERROR::SHADER_COMPILATION_ERROR del tipo: " <<
47                 tipo << "\n" << infoLog << "\n";
48         }
49     } else {
50         glGetProgramiv(shader, GL_LINK_STATUS, &success);
51         if (!success) {
52             glGetProgramInfoLog(shader, 1024, NULL, infoLog);
53             std::cerr << "ERROR::PROGRAM_LINKING_ERROR del tipo: " <<
54                 tipo << "\n" << infoLog << "\n";

```

```

53     }
54 }
55 }
56
57 // Funcion auxiliar: compilar y enlazar shaders
58 GLuint compilarShaders(const char* vs, const char* fs)
59 {
60     GLuint vid = glCreateShader(GL_VERTEX_SHADER);
61     glShaderSource(vid, 1, &vs, nullptr);
62     glCompileShader(vid);
63     chequearErroresShader(vid, "VERTEX");
64
65     GLuint fid = glCreateShader(GL_FRAGMENT_SHADER);
66     glShaderSource(fid, 1, &fs, nullptr);
67     glCompileShader(fid);
68     chequearErroresShader(fid, "FRAGMENT");
69
70     GLuint prog = glCreateProgram();
71     glAttachShader(prog, vid);
72     glAttachShader(prog, fid);
73     glLinkProgram(prog);
74     chequearErroresShader(prog, "PROGRAMA");
75
76     glDeleteShader(vid);
77     glDeleteShader(fid);
78     return prog;
79 }
80
81 // Construir matriz de escalamiento 2D (factores sx, sy)
82 void matrizEscalamiento(float sx, float sy, float M[9])
83 {
84     // OpenGL usa column-major: M[col*3 + row]
85     // col 0      col 1      col 2
86     M[0] = sx;    M[3] = 0.0f; M[6] = 0.0f; // fila 0
87     M[1] = 0.0f;  M[4] = sy;  M[7] = 0.0f; // fila 1
88     M[2] = 0.0f;  M[5] = 0.0f; M[8] = 1.0f; // fila 2
89 }
90
91 // Construir matriz de escalamiento alrededor de un punto (cx, cy)
92 void matrizEscalamientoAlrededorDe(float sx, float sy, float cx, float
    cy, float M[9])
93 {
94     // Matriz: T(cx,cy) * S(sx,sy) * T(-cx,-cy)
95     // Para un punto p: p' = S * (p - centro) + centro = S*p + (centro -
        S*centro)
96     float tx = cx - (sx * cx);
97     float ty = cy - (sy * cy);
98
99     M[0] = sx;    M[3] = 0.0f; M[6] = tx;
100    M[1] = 0.0f;   M[4] = sy;  M[7] = ty;
101    M[2] = 0.0f;   M[5] = 0.0f; M[8] = 1.0f;
102 }
103
104 // Construir matriz de escalamiento uniforme (mismo factor en X y Y)
105 void matrizEscalamientoUniforme(float s, float M[9])
106 {
107     matrizEscalamiento(s, s, M);
108 }

```

```

109
110 // -----
111 // Inicializacion de VAO/VBO con los vertices de una figura
112 // -----
113 void inicializarGeometria()
114 {
115     // Crear un cuadrado para visualizar mejor el escalamiento
116     float vertices[] = {
117         // Cuadrado centrado en el origen
118         -0.4f, 0.4f, // v rtice 0 (superior izquierdo)
119         0.4f, 0.4f, // v rtice 1 (superior derecho)
120         0.4f, -0.4f, // v rtice 2 (inferior derecho)
121         -0.4f, -0.4f, // v rtice 3 (inferior izquierdo)
122
123         // Una cruz interna para mejor referencia visual
124         0.0f, 0.4f,
125         0.0f, -0.4f,
126         -0.4f, 0.0f,
127         0.4f, 0.0f
128     };
129
130     // ndices para dibujar el cuadrado como dos tri ngulos
131     unsigned int indices[] = {
132         0, 1, 2, // Primer tri ngulo
133         0, 2, 3 // Segundo tri ngulo
134     };
135
136     glGenVertexArrays(1, &vao);
137     glGenBuffers(1, &vbo);
138
139     // Crear y configurar VBO para v rtices
140     glBindVertexArray(vao);
141     glBindBuffer(GL_ARRAY_BUFFER, vbo);
142     glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices,
143                 GL_STATIC_DRAW);
144
145     // Atributo 0: vec2 aPos
146     glVertexAttribPointer(0, 2, GL_FLOAT, GL_FALSE, 2 * sizeof(float),
147                           (void*)0);
148     glEnableVertexAttribArray(0);
149     glBindVertexArray(0);
150 }
151
152 // -----
153 // Programa principal
154 // -----
155 int main()
156 {
157     // 1. Inicializar GLFW
158     if (!glfwInit()) {
159         std::cerr << "Error al inicializar GLFW" << std::endl;
160         return -1;
161     }
162
163     // Configurar GLFW para usar OpenGL 3.3 Core Profile
164     glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
165     glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
166     glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);

```

```

165
166 // 2. Crear la ventana
167 GLFWwindow* window = glfwCreateWindow(600, 600, "Escalamiento 2D --
GLFW", NULL, NULL);
168 if (!window) {
169     std::cerr << "Error al crear la ventana GLFW" << std::endl;
170     glfwTerminate();
171     return -1;
172 }
173 glfwMakeContextCurrent(window);
174
175 // 3. Inicializar GLEW
176 glewExperimental = GL_TRUE;
177 if (glewInit() != GLEW_OK) {
178     std::cerr << "Error al inicializar GLEW" << std::endl;
179     return -1;
180 }
181
182 // Configuración inicial de OpenGL
183 glClearColor(0.15f, 0.15f, 0.15f, 1.0f); // fondo gris oscuro
184 glEnable(GL_BLEND);
185 glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
186
187 // 4. Compilar shaders y cargar geometría
188 shaderProg = compileShaders(verSrc, fragSrc);
189 inicializarGeometria();
190
191 // Ubicaciones de los uniforms
192 GLint locMat = glGetUniformLocation(shaderProg, "uTransform");
193 GLint locColor = glGetUniformLocation(shaderProg, "uColor");
194
195 float M[9];
196 float escalaX = 1.0f;
197 float escalaY = 1.0f;
198 float escalaUniforme = 1.0f;
199
200 std::cout << "=== CONTROLES ===" << std::endl;
201 std::cout << "Q / A: Aumentar/Disminuir escala en X" << std::endl;
202 std::cout << "W / S: Aumentar/Disminuir escala en Y" << std::endl;
203 std::cout << "E / D: Aumentar/Disminuir escala uniforme" << std::
endl;
204 std::cout << "R: Reiniciar escalas" << std::endl;
205 std::cout << "ESC: Salir" << std::endl;
206 std::cout << "=====" << std::endl;
207
208 // 5. Bucle principal de renderizado (Render Loop)
209 while (!glfwWindowShouldClose(window))
210 {
211     // Procesar entradas
212     if (glfwGetKey(window, GLFW_KEY_ESCAPE) == GLFW_PRESS)
213         glfwSetWindowShouldClose(window, true);
214
215     // Control de escala en X (Q = aumentar, A = disminuir)
216     if (glfwGetKey(window, GLFW_KEY_Q) == GLFW_PRESS)
217         escalaX += 0.01f;
218     if (glfwGetKey(window, GLFW_KEY_A) == GLFW_PRESS)
219         escalaX -= 0.01f;
220

```



```

221 // Control de escala en Y (W = aumentar, S = disminuir)
222 if (glfwGetKey(window, GLFW_KEY_W) == GLFW_PRESS)
223     escalaY += 0.01f;
224 if (glfwGetKey(window, GLFW_KEY_S) == GLFW_PRESS)
225     escalaY -= 0.01f;
226
227 // Control de escala uniforme (E = aumentar, D = disminuir)
228 if (glfwGetKey(window, GLFW_KEY_E) == GLFW_PRESS)
229     escalaUniforme += 0.01f;
230 if (glfwGetKey(window, GLFW_KEY_D) == GLFW_PRESS)
231     escalaUniforme -= 0.01f;
232
233 // Reiniciar escalas con la tecla R
234 if (glfwGetKey(window, GLFW_KEY_R) == GLFW_PRESS) {
235     escalaX = 1.0f;
236     escalaY = 1.0f;
237     escalaUniforme = 1.0f;
238 }
239
240 // Limitar valores mínimos para evitar inversión/colapso
241 if (escalaX < 0.1f) escalaX = 0.1f;
242 if (escalaY < 0.1f) escalaY = 0.1f;
243 if (escalaUniforme < 0.1f) escalaUniforme = 0.1f;
244
245 // Limpiar el buffer de color
246 glClear(GL_COLOR_BUFFER_BIT);
247
248 // Activar el programa shader
249 glUseProgram(shaderProg);
250
251 // --- Dibujar figura ORIGINAL (escala 1,1) en rojo ---
252 matrizEscalamiento(1.0f, 1.0f, M);
253 glUniformMatrix3fv(locMat, 1, GL_FALSE, M);
254 glUniform4f(locColor, 1.0f, 0.0f, 0.0f, 0.5f); // RGBA rojo
    semitransparente
255
256 glBindVertexArray(vao);
257 glLineWidth(2.0f);
258 glDrawArrays(GL_QUADS, 0, 4); // Dibujar cuadrado original
259 glDrawArrays(GL_LINES, 4, 4); // Dibujar cruz interna
260
261 // --- Dibujar figura ESCALADA en X e Y (alrededor del origen)
    en verde ---
262 matrizEscalamiento(escalaX, escalaY, M);
263 glUniformMatrix3fv(locMat, 1, GL_FALSE, M);
264 glUniform4f(locColor, 0.0f, 1.0f, 0.0f, 0.7f); // RGBA verde
    semitransparente
265 glDrawArrays(GL_QUADS, 0, 4);
266 glDrawArrays(GL_LINES, 4, 4);
267
268 // --- Dibujar figura con ESCALAMIENTO UNIFORME (alrededor del
    centro) en azul ---
269 matrizEscalamientoUniforme(escalaUniforme, M);
270 glUniformMatrix3fv(locMat, 1, GL_FALSE, M);
271 glUniform4f(locColor, 0.0f, 0.2f, 0.9f, 0.7f); // RGBA azul
    semitransparente
272 glDrawArrays(GL_QUADS, 0, 4);
273 glDrawArrays(GL_LINES, 4, 4);

```

```

274     glBindVertexArray(0);
275
276     // Mostrar valores de escala en consola cada cierto tiempo
277     static int frameCount = 0;
278     if (++frameCount % 60 == 0) {
279         std::cout << "Escala X: " << escalaX << " | Escala Y: " <<
280             escalaY
281             << " | Escala Uniforme: " << escalaUniforme << std
282                 ::endl;
283     }
284
285     // Intercambiar buffers y procesar eventos
286     glfwSwapBuffers(window);
287     glfwPollEvents();
288 }
289
290 // 6. Limpieza de memoria al salir
291 glDeleteVertexArrays(1, &vao);
292 glDeleteBuffers(1, &vbo);
293 glDeleteProgram(shaderProg);
294
295 glfwDestroyWindow(window);
296 glfwTerminate();
297
298 return 0;
299 }

```

Listing 1: Escalamiento 2D

3.2. Código C++ completo — Rotación

```

1  #include <GL/glew.h>
2  #include <GLFW/glfw3.h>
3  #include <glm/glm.hpp>
4  #include <glm/gtc/type_ptr.hpp>
5  #include <iostream>
6  #include <string>
7  #include <cmath>
8
9  // -----
10 // Fuentes de los shaders (en crudo como cadenas)
11 // -----
12 const char* vertSrc = R"(
13 #version 330 core
14 layout(location = 0) in vec2 aPos;
15 uniform mat3 uTransform;
16 void main() {
17     vec3 p = uTransform * vec3(aPos, 1.0);
18     gl_Position = vec4(p.xy, 0.0, 1.0);
19 }
20 )";
21
22 const char* fragSrc = R"(
23 #version 330 core
24 uniform vec4 uColor;
25 out vec4 fragColor;

```

```

26 void main() {
27     fragColor = uColor;
28 }
29 );
30
31 // -----
32 // Variables globales
33 // -----
34 GLuint shaderProg, vao, vbo;
35
36 // Funci n auxiliar para revisar errores de compilaci n
37 void chequearErroresShader(GLuint shader, std::string tipo) {
38     GLint success;
39     GLchar infoLog[1024];
40     if (tipo != "PROGRAMA") {
41         glGetShaderiv(shader, GL_COMPILE_STATUS, &success);
42         if (!success) {
43             glGetShaderInfoLog(shader, 1024, NULL, infoLog);
44             std::cerr << "ERROR::SHADER_COMPILATION_ERROR del tipo: " <<
45                 tipo << "\n" << infoLog << "\n";
46         }
47     } else {
48         glGetProgramiv(shader, GL_LINK_STATUS, &success);
49         if (!success) {
50             glGetProgramInfoLog(shader, 1024, NULL, infoLog);
51             std::cerr << "ERROR::PROGRAM_LINKING_ERROR del tipo: " <<
52                 tipo << "\n" << infoLog << "\n";
53         }
54     }
55 }
56
57 // Funcion auxiliar: compilar y enlazar shaders
58 GLuint compilarShaders(const char* vs, const char* fs)
59 {
60     GLuint vid = glCreateShader(GL_VERTEX_SHADER);
61     glShaderSource(vid, 1, &vs, nullptr);
62     glCompileShader(vid);
63     chequearErroresShader(vid, "VERTEX");
64
65     GLuint fid = glCreateShader(GL_FRAGMENT_SHADER);
66     glShaderSource(fid, 1, &fs, nullptr);
67     glCompileShader(fid);
68     chequearErroresShader(fid, "FRAGMENT");
69
70     GLuint prog = glCreateProgram();
71     glAttachShader(prog, vid);
72     glAttachShader(prog, fid);
73     glLinkProgram(prog);
74     chequearErroresShader(prog, "PROGRAMA");
75
76     glDeleteShader(vid);
77     glDeleteShader(fid);
78     return prog;
79 }
80
81 // Construir matriz de rotaci n 2D ( ngulo en grados)
82 void matrizRotacion(float anguloGrados, float M[9])
83 {

```

```

82     float rad = anguloGrados * M_PI / 180.0f;
83     float cosA = cos(rad);
84     float sinA = sin(rad);
85
86     // OpenGL usa column-major: M[col*3 + row]
87     //   col 0       col 1       col 2
88     M[0] = cosA;    M[3] = -sinA;  M[6] = 0.0f;  // fila 0
89     M[1] = sinA;    M[4] =  cosA;  M[7] = 0.0f;  // fila 1
90     M[2] = 0.0f;    M[5] =  0.0f;  M[8] = 1.0f;  // fila 2
91 }
92
93 // Construir matriz de rotación alrededor de un punto (cx, cy)
94 void matrizRotacionAlrededorDe(float anguloGrados, float cx, float cy,
95     float M[9])
96 {
97     float rad = anguloGrados * M_PI / 180.0f;
98     float cosA = cos(rad);
99     float sinA = sin(rad);
100
101     // Matriz: T(cx,cy) * R( ) * T(-cx,-cy)
102     // Para un punto p: p' = R * (p - centro) + centro = R*p + (centro -
103     // R*centro)
104     float tx = cx - (cosA * cx - sinA * cy);
105     float ty = cy - (sinA * cx + cosA * cy);
106
107     M[0] = cosA;    M[3] = -sinA;  M[6] = tx;
108     M[1] = sinA;    M[4] =  cosA;  M[7] = ty;
109     M[2] = 0.0f;    M[5] =  0.0f;  M[8] = 1.0f;
110 }
111
112 // -----
113 // Inicializacion de VAO/VBO con los vertices de una figura
114 // -----
115 void inicializarGeometria()
116 {
117     // Crear un tri ngulo m s interesante para ver la rotaci n
118     float vertices[] = {
119         // Tri ngulo
120         0.0f,  0.5f,    // v rtice 0 (superior)
121        -0.4f, -0.3f,    // v rtice 1 (inferior izquierdo)
122         0.4f, -0.3f,    // v rtice 2 (inferior derecho)
123
124         // Cuadrado (opcional, descomentar si se quiere)
125         // -0.3f,  0.3f,
126         //  0.3f,  0.3f,
127         //  0.3f, -0.3f,
128         // -0.3f, -0.3f
129     };
130
131     glGenVertexArrays(1, &vao);
132     glGenBuffers(1, &vbo);
133
134     glBindVertexArray(vao);
135     glBindBuffer(GL_ARRAY_BUFFER, vbo);
136     glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices,
137         GL_STATIC_DRAW);
138
139     // Atributo 0: vec2 aPos

```

```

137     glVertexAttribPointer(0, 2, GL_FLOAT, GL_FALSE, 2 * sizeof(float),
138         (void*)0);
139     glEnableVertexAttribArray(0);
140     glBindVertexArray(0);
141 }
142 // -----
143 // Programa principal
144 // -----
145 int main()
146 {
147     // 1. Inicializar GLFW
148     if (!glfwInit()) {
149         std::cerr << "Error al inicializar GLFW" << std::endl;
150         return -1;
151     }
152
153     // Configurar GLFW para usar OpenGL 3.3 Core Profile
154     glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
155     glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
156     glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);
157
158     // 2. Crear la ventana
159     GLFWwindow* window = glfwCreateWindow(600, 600, "Rotacion 2D -- GLFW",
160         NULL, NULL);
161     if (!window) {
162         std::cerr << "Error al crear la ventana GLFW" << std::endl;
163         glfwTerminate();
164         return -1;
165     }
166     glfwMakeContextCurrent(window);
167
168     // 3. Inicializar GLEW
169     glewExperimental = GL_TRUE;
170     if (glewInit() != GLEW_OK) {
171         std::cerr << "Error al inicializar GLEW" << std::endl;
172         return -1;
173     }
174
175     // Configuración inicial de OpenGL
176     glClearColor(0.15f, 0.15f, 0.15f, 1.0f); // fondo gris oscuro
177
178     // 4. Compilar shaders y cargar geometría
179     shaderProg = compileShaders(verSrc, fragSrc);
180     inicializarGeometria();
181
182     // Ubicaciones de los uniforms
183     GLint locMat = glGetUniformLocation(shaderProg, "uTransform");
184     GLint locColor = glGetUniformLocation(shaderProg, "uColor");
185
186     float M[9];
187     float angulo = 0.0f;
188
189     // 5. Bucle principal de renderizado (Render Loop)
190     while (!glfwWindowShouldClose(window))
191     {
192         // Procesar entradas
193         if (glfwGetKey(window, GLFW_KEY_ESCAPE) == GLFW_PRESS)

```

```

193         glfwSetWindowShouldClose(window, true);
194
195         // Rotar con las teclas R (derecha) y T (izquierda)
196         if (glfwGetKey(window, GLFW_KEY_R) == GLFW_PRESS)
197             angulo += 1.0f;
198         if (glfwGetKey(window, GLFW_KEY_T) == GLFW_PRESS)
199             angulo -= 1.0f;
200
201         // Limpiar el buffer de color
202         glClear(GL_COLOR_BUFFER_BIT);
203
204         // Activar el programa shader
205         glUseProgram(shaderProg);
206
207         // --- Dibujar figura ORIGINAL (sin rotación) en rojo ---
208         matrizRotacion(0.0f, M);
209         glUniformMatrix3fv(locMat, 1, GL_FALSE, M);
210         glUniform4f(locColor, 1.0f, 0.0f, 0.0f, 1.0f); // RGBA rojo
211
212         glBindVertexArray(vao);
213         glLineWidth(2.0f);
214         glDrawArrays(GL_TRIANGLES, 0, 3); // Dibujar triángulo
215
216         // --- Dibujar figura ROTADA (alrededor del origen) en verde ---
217         matrizRotacion(angulo, M);
218         glUniformMatrix3fv(locMat, 1, GL_FALSE, M);
219         glUniform4f(locColor, 0.0f, 1.0f, 0.0f, 0.7f); // RGBA verde
220             semitransparente
221         glDrawArrays(GL_TRIANGLES, 0, 3);
222
223         // --- Dibujar figura ROTADA alrededor de su centro (0,0) en
224             azul ---
225         // Nota: Para este triángulo, el centro aproximado está en
226             (0,0)
227         matrizRotacionAlrededorDe(angulo * 1.5f, 0.0f, 0.0f, M);
228         glUniformMatrix3fv(locMat, 1, GL_FALSE, M);
229         glUniform4f(locColor, 0.0f, 0.2f, 0.9f, 0.7f); // RGBA azul
230             semitransparente
231         glDrawArrays(GL_TRIANGLES, 0, 3);
232
233         glBindVertexArray(0);
234
235         // Mostrar ángulo en consola cada cierto tiempo
236         static int frameCount = 0;
237         if (++frameCount % 60 == 0) {
238             std::cout << " ángulo de rotación (verde): " << angulo << "
239                 grados" << std::endl;
240         }
241
242         // Intercambiar buffers y procesar eventos
243         glfwSwapBuffers(window);
244         glfwPollEvents();
245     }
246
247     // 6. Limpieza de memoria al salir
248     glDeleteVertexArrays(1, &vao);
249     glDeleteBuffers(1, &vb0);
250     glDeleteProgram(shaderProg);

```

```

246
247     glfwDestroyWindow(window);
248     glfwTerminate();
249
250     return 0;
251 }

```

Listing 2: Rotación 2D

4. COMPARATIVA: Pipeline Clásico vs. Pipeline Moderno

Característica	OpenGL Clásico	OpenGL Moderno
Transformaciones	<code>glTranslatef()</code>	Matriz uniform en GLSL
Envío de vértices	<code>glBegin/glEnd</code>	VAO + VBO
Versión OpenGL mín.	1.x	3.3+ core profile
Flexibilidad	Limitada (pipeline fijo)	Total (programable)
Rendimiento GPU	Bajo	Alto
Control del color	<code>glColor3f()</code>	<code>uniform vec4</code>
Reutilización shader	No aplica	Un shader, N objetos

5. ACTIVIDADES EN CLASE

1. Escribir un programa que escale un círculo dos veces su dimensión original manteniendo fijo su centro.
2. Realizar una traslación y luego una rotación de un rectángulo.

6. BIBLIOGRAFÍA

- Hearn, D. & Baker, M.P. (2006). *Gráficas por Computadora con OpenGL* (3^a ed.). Pearson Prentice Hall, Madrid.
- Elias, R. (2019). *Digital Media: A Problem-Solving Approach for Computer Graphics*. eBook, New York.
- Guha, S. (2019). *Computer Graphics Through OpenGL* (3^a ed.). Taylor & Francis Group, NW.
- Gordon, V.S. & Clevenger, J. (2019). *Computer Graphics Programming in OpenGL with C++*. Mercury Learning and Information.
- Shreiner, D. et al. (2013). *OpenGL Programming Guide* (8^a ed.). Addison-Wesley.
<https://www.opengl.org/documentation/>